

Konzeption: Statische Unternehmenswebsite in der Cloud

Cloud Programming (DLBSEPCP01_D) – Aufgabe 1, Konzeptionsphase

1. Szenario & Anforderungsanalyse

Ein international tätiges Unternehmen benötigt eine öffentliche Unternehmenswebsite, die als erste Anlaufstelle und Aushängeschild dient. Da Besucher weltweit und rund um die Uhr zugreifen, führt jeder Ausfall oder jede spürbare Verzögerung unmittelbar zu Reputations- und potenziellem Geschäftsverlust. Daraus folgt die Vorgabe einer **weltweit hochverfügbaren** Bereitstellung mit einem Ziel-SLA von $\geq 99,9\%$. Die Website selbst ist eine einfache statische HTML-Seite (Hello-World-Prototyp); bewertet wird nicht die Webentwicklung, sondern die robuste, skalierbare und reproduzierbare Cloud-Infrastruktur.

Funktionale Anforderungen: statisches Hosting einer HTML-Datei, Erreichbarkeit über HTTPS, eigene Custom Domain (`cloud.trappeonline.xyz`), globale Erreichbarkeit ohne spürbare Ladezeiten.

Nicht-funktionale Anforderungen: Hochverfügbarkeit ohne Single Point of Failure; automatische Skalierung von Lastspitzen ohne manuellen Eingriff; minimale Ladezeiten unabhängig vom Standort (Edge-Auslieferung); Schutz vor DDoS- und Layer-7-Angriffen (WAF); Kosteneffizienz durch serverlose bzw. managed Dienste (Pay-per-Use, keine Leerlaufkosten); vollständige Reproduzierbarkeit der Infrastruktur als Infrastructure-as-Code (IaC).

Rahmenbedingungen: Budget von 100 USD (Azure for Students). Azure Front Door und Azure CDN Classic sind für Student-Abos technisch gesperrt – eine zentrale Einschränkung, die die Architekturwahl direkt prägt.

Hinweis zur Skalierung: Da es sich um rein statische Inhalte handelt, existiert bewusst kein klassisches Backend. Die in der Aufgabe geforderte automatische Skalierung wird stattdessen durch den serverlosen Hosting-Dienst und die vorgelagerte CDN-Edge-Schicht übernommen, die Lastspitzen ohne Kapazitätsplanung abfangen.

2. Architekturentscheidung

Cloud-Anbieter – Microsoft Azure. AWS erfordert eine Kreditkarte zur Registrierung, Google Cloud bietet weniger passendes Kursmaterial. Azure stellt über *Azure for Students* Guthaben ohne Kreditkarte bereit und bietet einen passenden Service-Stack – daher als Hosting-Basis gewählt.

Verworfenne Ansätze. Der ursprüngliche Plan *Azure Storage + Azure Front Door* (globale PoPs, WAF, HTTPS) scheitert an der Sperre von Front Door für Student-Abos. Der Fallback *Azure CDN Classic* entfällt, da Microsoft die Erstellung neuer Classic-Profiles abgekündigt hat. *Azure Static Web Apps allein* ist verfügbar und kostenlos, bietet aber eine Black-Box-CDN-Schicht ohne konfigurierbare WAF und ohne granulare Kontrolle über Caching und Sicherheitsregeln.

Gewählte Architektur – Azure Static Web Apps + Cloudflare. Die finale Lösung kombiniert **Azure Static Web Apps** (Free Tier) als Hosting-Plattform mit **Cloudflare** (Free Plan) als vorgelagerter CDN- und Sicherheitsschicht. Cloudflare betreibt mit über 300 PoPs eines der größten CDN-Netzwerke weltweit und liefert WAF (OWASP Top 10), DDoS-Schutz auf Layer 3/4/7, automatische HTTPS-Terminierung sowie die DNS-Verwaltung der Custom Domain. So werden alle ursprünglichen Anforderungen erfüllt, obwohl Azure-native CDN-Dienste im Student-Abo nicht verfügbar sind. Die Hinzunahme von Cloudflare ist damit keine willkürliche Anbieterwahl, sondern eine technisch erzwungene, bewusst gewählte Ergänzung.

Sicherheit / Origin-Isolation. Damit Besucher nicht an Cloudflare vorbei direkt den `*.azurestaticapps.net`-Endpoint aufrufen und so die WAF umgehen, wird der Origin-Zugriff abgesichert (Beschränkung auf Cloudflare-Traffic). Secrets und Zugriffstoken werden nicht im IaC-Code hinterlegt, sondern über Variablen bzw. Umgebungswerte verwaltet.

Kosteneffizienz. Beide Dienste bleiben im kostenlosen Kontingent und erzeugen keine Leerlaufkosten; die Infrastruktur skaliert elastisch (hoch *und* runter). Nach dem Deployment-Test wird die Infrastruktur

per terraform destroy vollständig abgebaut.

3. Komponenten & Datenfluss

Azure Static Web Apps hostet die `index.html`, ist vollständig serverlos, hochverfügbar über die globale Azure-Infrastruktur und stellt einen öffentlichen HTTPS-Endpoint bereit. **Cloudflare** sitzt als Edge davor, leitet Besucher zum nächstgelegenen PoP, terminiert HTTPS, filtert Angriffe (WAF/DDoS) und verwaltet die Custom-Domain. **Terraform** verwaltet beide Provider (`azurerm` + `cloudflare`) und macht Resource Group, Static Web App und DNS-Record vollständig reproduzierbar.

Datenfluss: Eine HTTPS-Anfrage an `cloud.trappeonline.xyz` erreicht den nächstgelegenen Cloudflare-PoP. Bei Cache-Hit erfolgt die Auslieferung direkt vom Edge; die WAF prüft jede Anfrage und blockiert bösartige Muster. Nur bei Cache-Miss wird die Anfrage an den Azure-Origin weitergeleitet, der `index.html` zurückgibt – die Antwort fließt über den Cloudflare-Edge zum Besucher zurück.

[C4-/Komponenten-Diagramm hier einfügen]

Komponenten-Interaktion und Datenfluss (Besucher → Cloudflare Edge → Azure Static Web Apps).
Alle Abkürzungen ausgeschreiben, je Box: Cloud-Dienst + eigener Inhalt (z. B. `index.html`).